Complete Technical Blueprint
Production-Grade Architecture
Implementation-Ready
Specification

TECHNICAL BLUEPRINT

SynthForge AI Platform

A production-grade, venture-scale synthetic data generation platform architected to replicate and surpass Tonic Fabricate with provider-agnostic AI orchestration, multi-agent collaboration, and enterprise-grade infrastructure.

This document provides an exhaustive, implementation-ready specification covering product research, system architecture, AI orchestration layer, synthetic data engine, multi-agent system, enterprise features, database schema, UI/UX specifications, production code examples, and deployment strategy for a competitive SaaS platform.

Version 1.0
May 2026
Confidential

Z.AI ENGINEERING

Table of Contents

1. Executive Summary 4

2. Product Research and Reverse Engineering 4

2.1 Core Product Features . . . 4

2.2 Data Generation Capabilities . . . 5

2.3 Enterprise Architecture Analysis . . . 5

2.4 Inferred Architecture Decisions . . . 6

3. System Architecture 6

3.1 Frontend Architecture . . . 7

3.2 Backend Architecture . . . 7

3.3 Multi-Tenant Architecture . . . 8

4. AI Orchestration Layer 9

4.1 Provider Abstraction Interface . . . 9

4.2 Dynamic Model Routing . . . 9

4.3 Retry and Fallback Logic . . . 10

4.4 Structured Output Enforcement . . . 10

4.5 Prompt Versioning and Management . . . 10

5. Synthetic Data Engine 10

5.1 Generation Pipeline Architecture . . . 11

5.2 Supported Data Formats . . . 11

5.3 PII Detection and Data Masking . . . 11

5.4 Referential Integrity and Constraint Enforcement . . . 12

5.5 Context-Aware and Domain-Specific Generation . . . 12

5.6 Deterministic Generation and Reproducibility . . . 12

6. Multi-Agent AI System 13

6.1 Schema Analysis Agent . . . 13

6.2 Data Quality Agent . . . 13

6.3 Synthetic Generation Agent . . . 14
6.4 Validation Agent . . . 15
6.5 Optimization Agent . . . 15
6.6 Security/Compliance Agent . . . 15
6.7 Prompt Engineering Agent . . . 16
6.8 Agent Communication Protocol . . . 16
7. Enterprise Features 17
7.1 Authentication and SSO . . . 17
7.2 Role-Based Access Control . . . 17
7.3 Compliance and Audit Logging . . . 18
7.4 Billing and Subscription Management . . . 18
8. Database Schema and Infrastructure Design 19
8.1 Core Entity Tables . . . 19
8.2 Generation and AI Tables . . . 19
8.3 Billing and Audit Tables . . . 20
8.4 Infrastructure Architecture . . . 20
9. UI/UX Clone Specification 21
9.1 Landing Page . . . 21
9.2 Dashboard . . . 22
9.3 Dataset Creation Wizard . . . 22
9.4 AI Generation Studio (Chat Interface) . . . 22
9.5 Schema Editor . . . 22
9.6 Prompt Playground . . . 23
9.7 Team Management and Admin Dashboard . . . 23
10. Production Code Examples 23
10.1 AI Orchestration Layer . . . 23
10.2 Synthetic Data Pipeline . . . 24
10.3 Multi-Agent Coordination . . . 25

10.4 Authentication and RBAC 26
10.5 Streaming Architecture and WebSocket Updates 27
10.6 Schema Inference Engine 28
11. Deployment and Monetization Strategy 29
11.1 SaaS Deployment Architecture 29
11.2 Pricing Model 29
11.3 Usage Metering and Cost Optimization 30
11.4 Scaling Roadmap 30
12. Final Deliverables and Project Plan 30
12.1 File and Folder Structure 31
12.2 MVP Plan (Phase 1: Months 1-6) 31
12.3 Production Scaling Plan 31

1. Executive Summary

SynthForge AI is a production-grade, venture-scale synthetic data generation platform designed to replicate and surpass the capabilities of Tonic Fabricate. This technical blueprint provides an exhaustive, implementation-ready specification for building a highly scalable AI-powered synthetic data platform from the ground up. The platform leverages a provider-agnostic AI orchestration layer that abstracts away underlying LLM providers (OpenAI, Claude, DeepSeek, and future providers) so that end users experience a fully proprietary AI system. The architecture is designed for multi-tenant SaaS deployment with enterprise-grade security, compliance, and horizontal scalability.

The core innovation of SynthForge AI lies in its multi-agent orchestration system that combines specialized AI agents for schema analysis, data quality assessment, synthetic generation, validation, optimization, security compliance, and prompt engineering. These agents collaborate through a structured communication protocol to deliver high-fidelity synthetic data that maintains referential integrity, statistical accuracy, and domain-specific realism. The platform supports a comprehensive range of data formats including relational databases (PostgreSQL, MySQL, Oracle, SQL Server, Databricks), structured files (CSV, JSON, XML, SQLite, JSONL), and unstructured documents (PDF, DOCX, PPTX, EML).

The frontend is built on Next.js 15+ with TypeScript, TailwindCSS, and shadcn/ui, delivering a chat-first conversational interface that mirrors the Tonic Fabricate UX paradigm. The backend runs on FastAPI with PostgreSQL, Redis, and a vector database, supporting WebSocket real-time updates, queue-based job processing, and multi-tenant isolation. The entire system is containerized with Docker and deployable on Kubernetes for enterprise self-hosting or managed SaaS. This blueprint covers product research, system architecture, AI layer design, synthetic data engine, multi-agent system, enterprise features, database schema, infrastructure, UI/UX specifications, production code examples, and deployment strategy.

2. Product Research and Reverse Engineering

Tonic Fabricate, originally created by Mark Brocato (founder of Mockaroo) and acquired by Tonic.ai in April 2025, represents the state of the art in AI-powered synthetic data generation. The platform positions itself as "the AI agent for synthetic data generation," enabling users to create realistic, privacy-safe data from scratch without requiring any pre-existing production data. The Fabricate Data Agent reached general availability in November 2025, marking a significant shift from rule-based to AI-native data generation. This section provides a comprehensive analysis of the product, its architecture, and inferred implementation decisions based on publicly available information.

2.1 Core Product Features

The central feature of Tonic Fabricate is the Data Agent, an AI-powered conversational interface that interprets natural language prompts, uploaded schemas, or existing sample data to generate high-fidelity synthetic datasets. The agent leverages Anthropic Claude models (Sonnet 4.5, Opus 4.7, Haiku 4.5) combined with Tonic.ai proprietary synthetic data generators. Users interact exclusively through a chat-based UI where they describe their data needs conversationally, and the agent generates data in real-time with iterative refinement capabilities. Each agent action produces either SQL or JavaScript that users can view and copy, providing transparency into the generation process. A revert capability allows users to undo any change made by the agent, creating a safe experimentation environment.

The platform supports two primary generation paradigms: schema-first generation, where users upload empty schemas (SQL DDL, Prisma, Swagger YAML) and the agent populates them with appropriate data; and seed-data enhancement, where users provide real data samples (CSV, Excel) that serve as reference material without being exposed in the output. Schema modification is possible through natural language, allowing users to add columns, tables, or change data types conversationally. The agent understands table relationships, foreign keys, and constraints, ensuring that generated data maintains referential integrity across complex multi-table schemas.

2.2 Data Generation Capabilities

Tonic Fabricate supports a comprehensive range of data formats for both input and output. For structured data, the platform generates relationally intact databases for PostgreSQL, MySQL, Databricks, Oracle, and SQL Server, as well as file-based formats including CSV, SQLite, JSONL, XML, and nested JSON structures. For unstructured data, the system generates free-text content populated from synthetic structured data, with export to PDF, DOCX, PPTX, and EML formats. The PDF generation pipeline uses an isolated pdf-sidecar container with no network access for security. Mock REST API endpoints can be created from Swagger/OpenAPI specs or via natural language, with auto-generated OpenAPI specifications, interactive testing, request history, and authentication headers.

The platform includes automated workflow capabilities for scheduled or parameterized data generation, with API-driven execution, callback progress tracking, and downloadable results. Custom Skills allow users to define reusable patterns for the Data Agent, and Plan Mode (available on Plus and Enterprise plans) enables users to review proposed changes before execution. Live Connect allows direct generation into external databases, and database profiling analyzes existing database structures to inform synthetic data generation. The pricing model follows a three-tier structure (Free at $10/month AI credits, Plus at $29/month with $25 AI credits, and Enterprise with custom pricing), with AI usage metered at Anthropic token cost plus a $0.50 per million tokens processing fee.

2.3 Enterprise Architecture Analysis

Based on publicly available deployment documentation and architecture references, Tonic Fabricate runs as a Docker-compatible container service with a specific set of infrastructure components. The web container serves the REST API and Web Console, exposed via a load balancer with TLS termination. The pdf-sidecar container handles isolated PDF generation without network access. An application

worker container processes background and scheduled tasks. PostgreSQL 16+ serves as the application database for accounts, workspaces, users, database connections, and chat history. Redis 7+ provides temporary caching and task queue functionality. The system supports optional S3-compatible object storage for generated datasets and PDFs, with compatibility across Amazon S3, Google Cloud Storage, Azure Blob, Backblaze B2, and Cloudflare R2.

The security architecture implements a three-tier model (Presentation, Application, Data) with SOC 2 Type II certification, HIPAA administrative and technical controls, GDPR standard contractual controls, and CCPA compliance. Encryption uses AES-256 at rest and TLS 1.2/1.3 in transit, with application-level encryption for stored files. The infrastructure includes Web Application Firewalls with threat intelligence, intrusion detection software, audit logging of admin access, annual penetration testing, SAST and dependency scanning in the SDLC, and data loss prevention software. Enterprise features include SSO/SAML via OIDC Authorization Code Flow, RBAC with workspace-level permissions, account groups, and allowed domains for access control. Self-hosting is available for Enterprise licenses on AWS, Azure, GCP, custom VPS, or bare-metal infrastructure.

2.4 Inferred Architecture Decisions

Several architectural decisions can be inferred from the product behavior and documentation. The chat-first interface suggests a WebSocket-based real-time communication layer with server-sent events for streaming agent responses. The ability to view generated SQL/JavaScript implies that the agent uses a tool-calling pattern where the LLM generates database operations as structured tool calls rather than raw text. The revert capability suggests an event-sourcing or command pattern where each agent action is stored as a reversible command with before/after state. The isolated PDF generation container indicates a security-conscious architecture where untrusted content processing occurs in sandboxed environments with no network access.

The multi-turn conversation model with token tracking implies a conversation state management system that maintains full chat history in the database, sends relevant context with each LLM call, and tracks cumulative token usage per conversation and per account. The BYO LLM key feature (Enterprise only) suggests an abstraction layer where LLM provider configuration is injected at the account or workspace level rather than hardcoded. The workflow automation with API execution and callbacks indicates an asynchronous job processing system with webhook support. The pricing model with per-token metering and configurable overage limits requires a real-time usage tracking system that integrates with the billing infrastructure.

3. System Architecture

The SynthForge AI platform architecture is designed as a distributed, microservices-oriented system with clear separation of concerns across frontend, backend, AI orchestration, and infrastructure layers. The architecture prioritizes horizontal scalability, multi-tenant isolation, real-time responsiveness, and

enterprise-grade security. Every component is containerized and orchestrated via Kubernetes, with stateless application services that can be independently scaled based on demand. The system supports both SaaS deployment and enterprise self-hosting scenarios.

3.1 Frontend Architecture

The frontend is built on Next.js 15+ with the App Router, leveraging React Server Components for initial page loads and Client Components for interactive features. TypeScript provides type safety across the entire codebase, while TailwindCSS 4 and shadcn/ui deliver a consistent, accessible design system. The chat-first interface uses a custom streaming architecture built on the Vercel AI SDK patterns, with Server-Sent Events for real-time token delivery and WebSocket connections for bidirectional communication (job status, collaborative editing, notifications).

State management follows a layered approach: React Context for theme and authentication state, Zustand for complex client-side state (chat history, dataset previews, schema editors), and TanStack Query for server state with optimistic updates and background refetching. The dataset visualizer uses a virtualized table component capable of rendering millions of rows with constant memory usage, combined with a schema editor that provides real-time validation and intelligent autocomplete powered by the AI layer. The workflow builder implements a node-based visual editor with drag-and-drop support, using React Flow for the canvas and a custom execution engine for workflow orchestration.

Component	Technology	Purpose
Chat Interface	Next.js + AI SDK	Primary user interaction via conversational AI
Dataset Visualizer	Virtualized Table (AG Grid)	Render millions of rows with constant memory
Schema Editor	Monaco Editor + JSON Schema	Real-time validation, autocomplete, syntax highlighting
Prompt Playground	Custom Editor + Streaming	Iterative prompt testing with live output preview
Workflow Builder	React Flow + Custom Engine	Visual node-based workflow orchestration
Dashboard	Recharts + TanStack Query	Usage analytics, billing, team management
Export Pipeline	Web Workers + Streams API	Client-side export with progress tracking

Table 1: Frontend Component Architecture

3.2 Backend Architecture

The backend is built on FastAPI with a modular service architecture. Each domain (auth, projects, datasets, generation, agents, billing) is implemented as an independent service module with its own router, service layer, and data access layer. The API gateway handles authentication, rate limiting, request validation, and routing. PostgreSQL serves as the primary relational database with SQLAlchemy as the ORM and Alembic for migrations. Redis provides caching, session management, and the task queue backend for Celery workers. A Qdrant vector database stores embeddings for semantic search

over schemas, prompts, and generated datasets, enabling intelligent context retrieval for the AI agents.

The WebSocket architecture supports real-time updates for generation progress, collaborative editing, and notification delivery. Each WebSocket connection is authenticated via JWT token and associated with a specific workspace and user context. The system uses a pub/sub pattern over Redis channels for cross-instance message delivery, ensuring that WebSocket messages reach the correct client regardless of which application instance holds the connection. The queue system uses Celery with Redis as the broker, supporting priority queues, scheduled tasks, and retry logic with exponential backoff. Long-running generation jobs are processed by dedicated worker pods that can be independentlyently scaled based on queue depth.

Service	Technology	Responsibility
API Gateway	FastAPI + Traefik	Auth, rate limiting, routing, request validation
Auth Service	FastAPI + OAuth2	JWT, SSO/SAML, RBAC, API key management
Project Service	FastAPI + SQLAlchemy	Workspace/project/database CRUD operations
Generation Service	FastAPI + Celery	Synthetic data generation orchestration
Agent Service	FastAPI + AI SDK	Multi-agent coordination and tool execution
Schema Service	FastAPI + JSON Schema	Schema parsing, validation, inference
Export Service	FastAPI + Workers	Multi-format data export and delivery
Billing Service	FastAPI + Stripe	Usage metering, subscriptions, invoicing
Notification Service	FastAPI + WebSocket	Real-time updates, email, webhooks

Table 2: Backend Service Architecture

3.3 Multi-Tenant Architecture

The multi-tenant system uses a shared database with tenant isolation at the application layer. Each workspace maps to a tenant, and all database queries include a workspace_id filter enforced at the ORM level through a custom SQLAlchemy session mixin. Row-Level Security (RLS) policies in PostgreSQL provide defense-in-depth, ensuring that even if the application layer fails to filter, the database prevents cross-tenant data access. Connection pooling is managed per-tenant with configurable limits, and tenant-specific configuration (LLM provider, storage backend, SSO settings) is stored in a tenant_metadata table with encrypted sensitive fields.

The caching layer implements tenant-aware cache keys that include the workspace_id prefix, preventing cache leakage between tenants. Background jobs are tagged with tenant context and executed in isolated worker pools for enterprise tenants, while shared pools handle free and plus tier workloads with fair-share scheduling. The vector database partitions collections by tenant with separate embedding spaces, ensuring that semantic search results never cross tenant boundaries. File storage uses

tenant-prefixed bucket paths with IAM policies that enforce access control at the infrastructure level.

4. AI Orchestration Layer

The AI orchestration layer is the most critical component of SynthForge AI, providing a provider-agnostic abstraction that supports OpenAI, Claude, DeepSeek, and future LLM providers without exposing the underlying provider to end users. The system appears to users as a fully proprietary AI platform, with all provider-specific details hidden behind a unified interface. The orchestration layer handles model routing, cost optimization, retry and fallback logic, prompt versioning, structured output enforcement, JSON schema validation, agent memory, tool calling, multi-agent collaboration, and streaming token delivery.

4.1 Provider Abstraction Interface

The provider abstraction follows the Strategy pattern, where each LLM provider implements a common interface that normalizes request and response formats. The LLMProvider abstract base class defines methods for chat completion, streaming completion, structured output, token counting, and embedding generation. Each concrete provider (OpenAI, Claude, DeepSeek) implements these methods with provider-specific API calls while normalizing the output to a standardized format. The provider registry maintains a catalog of available providers with their capabilities, pricing, rate limits, and health status, enabling intelligent routing decisions.

4.2 Dynamic Model Routing

The routing engine evaluates multiple factors when selecting which provider and model to use for each request: task complexity (simple corrections vs. complex schema generation), cost constraints (per-tenant budget limits and overage thresholds), latency requirements (streaming responses require low time-to-first-token), provider health (circuit breaker pattern with configurable failure thresholds), and model capabilities (some tasks require structured output or tool calling support). The routing configuration is stored per-tenant, allowing enterprise customers to specify provider preferences or bring their own API keys for specific providers.

Cost optimization is achieved through a tiered routing strategy. Simple tasks like data corrections or small schema modifications are routed to cost-effective models (e.g., DeepSeek or Claude Haiku), while complex multi-table schema generation or domain-specific data synthesis uses high-capability models (e.g., Claude Opus or GPT-4o). The system maintains real-time cost tracking per conversation and per account, with configurable alerts when usage approaches billing thresholds. A prompt caching layer reduces costs by identifying semantically similar prompts and reusing cached responses when the underlying data has not changed.

4.3 Retry and Fallback Logic

The retry system implements exponential backoff with jitter for transient failures (rate limits, timeouts, server errors), with configurable maximum retries per provider. When a provider fails persistently, the fallback chain automatically routes requests to the next available provider in the priority order. Each provider has a circuit breaker that opens after a configurable number of consecutive failures, preventing cascading failures and allowing the provider time to recover. The circuit breaker transitions through three states: Closed (normal operation), Open (all requests fail-fast), and Half-Open (limited requests allowed to test recovery). All retry and fallback events are logged for observability and cost tracking, ensuring that failed requests on one provider are not double-billed when retried on another.

4.4 Structured Output Enforcement

Synthetic data generation requires precise, structured output from LLMs. The orchestration layer enforces structured output through multiple mechanisms: JSON schema validation with automatic retry on malformed output, function calling or tool use APIs where available (forcing the LLM to generate structured tool call parameters), post-processing pipelines that repair common LLM output errors (missing quotes, truncated JSON, incorrect data types), and few-shot prompting with examples of correctly formatted output. For providers that support structured output natively (OpenAI response_format, Claude tool_use), the system uses native support. For providers without native support, a validation and repair pipeline ensures output conforms to the expected schema.

4.5 Prompt Versioning and Management

All prompts used by the AI agents are versioned and stored in the database, enabling A/B testing, rollback, and audit trails. Each prompt version includes the template text, variable definitions, model configuration (temperature, max_tokens, top_p), and performance metrics (success rate, average token usage, user satisfaction scores). The prompt management system supports template variables with type checking, conditional sections, and dynamic context injection. Agent instructions at the workspace and database level are merged with system prompts at runtime, allowing per-tenant customization of agent behavior without modifying the base prompt templates.

5. Synthetic Data Engine

The synthetic data engine is the core value proposition of SynthForge AI, responsible for generating high-fidelity, privacy-safe data across a comprehensive range of formats and domains. The engine combines AI-powered generation with deterministic constraint enforcement, ensuring that generated data is both realistic and structurally valid. The architecture supports both batch generation for large datasets and streaming generation for real-time data creation, with configurable quality levels that balance generation speed against data fidelity.

5.1 Generation Pipeline Architecture

The generation pipeline follows a five-stage process: Schema Analysis, Data Planning, Synthetic Generation, Validation, and Export. In the Schema Analysis stage, the engine parses the input schema (SQL DDL, Prisma, Swagger YAML, CSV headers, or natural language description) and builds an internal representation that captures tables, columns, data types, constraints, foreign keys, and relationships. The Data Planning stage uses the Schema Analysis Agent to determine generation order (respecting foreign key dependencies), identify domain-specific data patterns, and select appropriate generators for each column. The Synthetic Generation stage executes the generation plan using a combination of AI-powered generation (for complex, domain-specific data) and deterministic generators (for structured data like names, addresses, and dates). The Validation stage runs a comprehensive suite of checks including referential integrity, constraint satisfaction, statistical distribution analysis, and PII detection. The Export stage transforms the generated data into the requested output format.

5.2 Supported Data Formats

Category	Formats	Generation Method
Relational DBs	PostgreSQL, MySQL, Oracle, SQL Server, Databricks	DDL parsing + constraint-aware generation
File-based Structured	CSV, JSON, JSONL, XML, SQLite	Schema inference + format-specific serialization
Nested Schemas	Nested JSON, GraphQL responses, API payloads	Recursive schema traversal + nested generation
Unstructured Docs	PDF, DOCX, PPTX, EML	Template-based population from structured data
SQL Dumps	.sql files with INSERT statements	Schema-aware SQL generation with escaping
API Mocks	OpenAPI/Swagger specs with mock endpoints	Schema extraction + endpoint stub generation

Table 3: Supported Data Formats and Generation Methods

5.3 PII Detection and Data Masking

The PII detection system uses a multi-layered approach combining regex pattern matching, Named Entity Recognition (NER) models, and AI-powered contextual analysis. The regex layer detects structured PII patterns (SSN, credit card numbers, phone numbers, email addresses) with high precision. The NER layer uses a fine-tuned transformer model to detect unstructured PII (person names, addresses, medical record numbers) that do not follow regular patterns. The AI layer provides contextual analysis, understanding that a 9-digit number in a medical context is likely an MRN while the same number in a financial context might be a SSN. When PII is detected in seed data, the system applies context-aware masking that preserves the format and statistical properties of the original data while replacing sensitive

values with synthetic equivalents. For example, real names are replaced with culturally appropriate synthetic names that maintain the same distribution of ethnic origins, and real addresses are replaced with geographically consistent synthetic addresses.

5.4 Referential Integrity and Constraint Enforcement

Maintaining referential integrity across generated data is one of the most challenging aspects of synthetic data generation. The engine uses a topological sort of the foreign key dependency graph to determine generation order, ensuring that parent records are always generated before child records. For circular foreign key references, the engine uses a deferred constraint resolution strategy where placeholder values are inserted first and updated after all records in the cycle are generated. Unique constraints are enforced through deterministic generators that produce unique values based on sequence counters, while check constraints are validated against generated values with automatic regeneration for violations. Composite foreign keys are handled by generating the parent record first and propagating the key values to all child records, ensuring consistency across the entire dataset.

5.5 Context-Aware and Domain-Specific Generation

The context-aware generation system uses AI to understand the semantic meaning of data columns and generate appropriate values. For example, when generating a medical dataset, the system understands that a "diagnosis_code" column should contain valid ICD-10 codes, that a "medication" column should contain real drug names, and that the relationship between diagnoses and medications should be medically plausible. This domain intelligence is achieved through a combination of few-shot prompting with domain-specific examples, RAG (Retrieval-Augmented Generation) over a curated knowledge base of domain schemas, and fine-tuned generation templates for common industries (healthcare, finance, e-commerce, SaaS, IoT, manufacturing). The system also supports custom domain definitions where users can specify domain-specific rules, vocabularies, and relationships through natural language instructions or structured configuration files.

5.6 Deterministic Generation and Reproducibility

All generation operations support deterministic output through seed-based random number generators. When a seed value is provided, the same input schema and configuration will always produce the same output data, enabling reproducible data generation for testing and compliance purposes. The deterministic generator uses a seeded PRNG (Pseudo-Random Number Generator) for all random decisions in the generation pipeline, including value selection, distribution sampling, and null value placement. Dataset versioning tracks the seed, schema, and configuration used for each generation, allowing exact reproduction of any historical dataset. This is critical for regulated industries where audit trails must demonstrate that test data was generated consistently and without bias.

6. Multi-Agent AI System

The SynthForge AI multi-agent system is designed around a specialized agent architecture where each agent has a clearly defined role, set of tools, and communication protocol. The agents collaborate through a shared message bus with structured message types, enabling complex multi-step generation workflows that require expertise from multiple domains. The orchestrator agent manages the overall workflow, delegating tasks to specialized agents based on the current generation stage and aggregating their results into a coherent output.

6.1 Schema Analysis Agent

Responsibilities: The Schema Analysis Agent parses and interprets all input schema formats, builds the internal schema representation, identifies relationships and dependencies, and generates the data generation plan. It supports SQL DDL scripts (including PostgreSQL, MySQL, Oracle, SQL Server dialects), Prisma schema files, Swagger/OpenAPI YAML specifications, CSV/Excel header inference, JSON Schema definitions, and natural language schema descriptions. The agent uses a combination of deterministic parsers for structured formats and AI-powered inference for ambiguous or incomplete schemas.

Prompt Design: The system prompt instructs the agent to analyze schemas methodically, identifying primary keys, foreign keys, unique constraints, check constraints, default values, and data type semantics. The prompt includes few-shot examples of schema analysis across different formats and domains, with particular emphasis on understanding the semantic meaning of column names and data types (e.g., distinguishing between a "price" column that should contain realistic product prices vs. a "discount_percent" column that should be bounded 0-100). The agent outputs a structured JSON representation of the analyzed schema with generation recommendations.

Tool Access: SQL parser (antlr4-based), JSON Schema validator, Prisma schema parser, Swagger/OpenAPI parser, file upload handler, schema diff calculator, dependency graph builder. **Memory Strategy:** Short-term memory holds the current schema being analyzed and intermediate parsing results. Long-term memory stores previously analyzed schemas and their generation plans for pattern reuse. **Failure Recovery:** If parsing fails, the agent falls back to AI-powered schema inference from sample data or natural language description. Partial schemas are accepted with the agent requesting clarification for ambiguous elements.

6.2 Data Quality Agent

Responsibilities: The Data Quality Agent validates generated data against the schema constraints, statistical distributions, and domain-specific rules. It detects anomalies, inconsistencies, and quality issues that may not violate explicit constraints but produce unrealistic or implausible data. The agent maintains quality metrics across generation runs and provides recommendations for improving data quality. It checks referential integrity (all foreign keys resolve to valid parent records), constraint

compliance (unique, check, not-null constraints are satisfied), statistical fidelity (generated data distributions match specified or inferred distributions), domain consistency (values are semantically appropriate for their column context), and edge case coverage (rare but valid values are represented in proportion to their expected frequency).

Prompt Design: The system prompt frames the agent as a meticulous data quality auditor that evaluates generated data against multiple quality dimensions. The prompt includes quality scoring rubrics and examples of common quality issues with their severity classifications. The agent outputs a structured quality report with scores per dimension, specific issues found, and recommendations for remediation. **Tool Access:** Constraint validator, statistical distribution analyzer, referential integrity checker, domain-specific rule engine, anomaly detection model, quality score calculator. **Memory Strategy:** Maintains a quality baseline for each schema and dataset, tracking improvement or degradation over time. Stores common quality issues and their resolutions for proactive prevention. **Failure Recovery:** If quality validation fails, the agent identifies the specific records and columns responsible and recommends targeted regeneration for those elements only, avoiding full dataset regeneration when possible.

6.3 Synthetic Generation Agent

Responsibilities: The Synthetic Generation Agent is the primary data production engine, responsible for generating realistic synthetic records based on the schema analysis and generation plan. It uses a combination of AI-powered generation for complex domain-specific data and deterministic generators for structured data types. The agent handles both batch generation (producing large datasets in a single operation) and streaming generation (producing records incrementally with real-time delivery). It manages generation state to ensure consistency across multi-table datasets and supports iterative refinement based on user feedback through the chat interface.

Prompt Design: The system prompt instructs the agent to generate data that is both structurally valid and semantically realistic. The prompt includes the full schema context (tables, relationships, constraints), the generation plan from the Schema Analysis Agent, domain-specific examples and rules, and any user-specified instructions or custom skills. The prompt emphasizes maintaining referential integrity, producing diverse and realistic distributions, avoiding obvious patterns or repetitions, and respecting data type semantics. The agent outputs structured tool calls (SQL INSERT statements, JSON objects, or API calls) rather than raw text, ensuring that generated data is immediately usable.

Tool Access: SQL executor, JSON serializer, CSV writer, file generator (PDF, DOCX, PPTX, EML), database connection manager, batch processor, stream writer, distribution sampler. **Memory Strategy:** Maintains the current generation state including generated primary keys, foreign key mappings, and sequence counters. Uses working memory to track generation progress and ensure consistency across multi-table generation. **Failure Recovery:** If generation fails mid-batch, the agent can resume from the last checkpoint without regenerating already-completed records. Constraint violations trigger targeted regeneration of specific records rather than full batch restart.

6.4 Validation Agent

Responsibilities: The Validation Agent performs comprehensive post-generation validation that goes beyond the Data Quality Agent real-time checks. It executes full dataset validation including cross-table consistency checks, statistical distribution comparison with target distributions, PII leakage detection (ensuring no real PII from seed data appears in output), format compliance verification (CSV quoting, JSON schema conformance, SQL syntax correctness), and data volume verification (correct number of records per table). The agent produces a detailed validation report that serves as an audit artifact for compliance purposes.

Prompt Design: The system prompt frames the agent as a compliance-focused validator that verifies data against both explicit requirements (schema constraints, user specifications) and implicit expectations (realistic distributions, no PII leakage, proper formatting). The agent outputs a structured validation report with pass/fail status per check, detailed findings, and remediation recommendations. **Tool Access:** Full dataset scanner, PII detection model, statistical testing framework, format validator, cross-reference checker, compliance report generator. **Memory Strategy:** Maintains a validation history per dataset, tracking which checks have been performed and their results. Stores PII detection patterns for continuous improvement. **Failure Recovery:** Validation failures trigger automatic remediation workflows that target the specific failing records or columns, with escalation to the Synthetic Generation Agent for complex issues.

6.5 Optimization Agent

Responsibilities: The Optimization Agent monitors generation performance and cost, optimizing the generation pipeline for efficiency without sacrificing data quality. It analyzes token usage patterns, identifies opportunities to use deterministic generators instead of AI-powered generation for simple data types, implements prompt caching for repeated generation patterns, and recommends batch sizes that balance throughput against memory constraints. The agent also manages model routing decisions, recommending cost-effective models for simple tasks while reserving high-capability models for complex generation scenarios.

Prompt Design: The system prompt frames the agent as a performance engineer that balances cost, speed, and quality. The prompt includes cost models for each LLM provider, performance benchmarks for deterministic generators, and quality thresholds that must be maintained. The agent outputs optimization recommendations with estimated cost and performance impact. **Tool Access:** Token usage tracker, cost calculator, performance profiler, cache manager, model router, benchmark runner. **Memory Strategy:** Maintains a performance baseline for each generation type and model combination. Tracks cost trends over time and identifies optimization opportunities proactively. **Failure Recovery:** If optimization leads to quality degradation (detected by the Data Quality Agent), the Optimization Agent automatically reverts to the previous configuration and adjusts its quality-cost trade-off model.

6.6 Security/Compliance Agent

Responsibilities: The Security and Compliance Agent ensures that all generated data complies with applicable regulations (GDPR, HIPAA, CCPA, SOC 2) and organizational policies. It scans seed data for PII before processing, validates that generated output contains no real PII, enforces data retention policies, manages encryption at rest and in transit, and produces compliance audit logs. The agent also handles data subject access requests (DSARs) by tracking which seed data influenced which generated records, enabling precise deletion when required.

Prompt Design: The system prompt frames the agent as a compliance officer that evaluates data against regulatory requirements and organizational policies. The prompt includes regulatory checklists for GDPR, HIPAA, and CCPA, with specific criteria for each regulation. The agent outputs compliance status per regulation, specific findings, and remediation actions. **Tool Access:** PII scanner, compliance rule engine, encryption manager, audit log writer, retention policy enforcer, DSAR handler. **Memory Strategy:** Maintains a compliance state per dataset that tracks which regulations apply, which checks have been performed, and the current compliance status. Stores regulatory rule updates for continuous compliance. **Failure Recovery:** Compliance violations trigger immediate data quarantine, preventing export or delivery until the violation is resolved. The agent generates remediation plans and can automatically regenerate non-compliant data.

6.7 Prompt Engineering Agent

Responsibilities: The Prompt Engineering Agent manages and optimizes the prompts used by all other agents. It analyzes prompt performance metrics (success rate, quality score, token efficiency), identifies opportunities for prompt improvement, generates and tests prompt variants through A/B testing, and maintains the prompt version control system. The agent also helps users craft effective custom instructions by translating natural language requirements into structured prompt additions that improve generation quality without increasing token costs unnecessarily.

Prompt Design: The system prompt frames the agent as a prompt optimization specialist that iteratively improves prompt templates based on empirical performance data. The prompt includes evaluation criteria (output quality, token efficiency, user satisfaction), A/B testing methodology, and constraints for prompt modifications (no changes to safety instructions, no removal of schema context). The agent outputs modified prompt templates with rationale and expected impact. **Tool Access:** Prompt template manager, A/B test runner, quality evaluator, token counter, version control system. **Memory Strategy:** Maintains a prompt performance history that tracks the impact of each modification on quality and cost metrics. Stores successful prompt patterns for reuse across similar generation scenarios. **Failure Recovery:** If a prompt modification leads to degraded performance, the agent automatically rolls back to the previous version and adjusts its optimization strategy.

6.8 Agent Communication Protocol

Agents communicate through a structured message bus with typed messages. Each message includes a sender agent ID, recipient agent ID (or broadcast), message type (request, response, notification, error), payload (structured JSON), and correlation ID for tracking multi-step workflows. The Orchestrator

Agent manages the overall workflow by dispatching tasks to specialized agents, collecting their results, and making decisions about next steps based on the aggregated outcomes. The communication protocol supports both synchronous request-response patterns (for sequential operations like schema analysis followed by generation planning) and asynchronous notification patterns (for parallel operations like concurrent quality validation and compliance checking).

7. Enterprise Features

Enterprise features are critical for adoption in regulated industries and large organizations. SynthForge AI implements a comprehensive enterprise feature set that addresses authentication, authorization, compliance, auditing, billing, team collaboration, and administrative control. All enterprise features are designed with SOC 2 Type II compliance in mind, following the principle of least privilege and defense-in-depth security.

7.1 Authentication and SSO

The authentication system supports multiple identity providers through OIDC (OpenID Connect) and SAML 2.0 protocols. Built-in authentication uses email/password with bcrypt hashing and optional multi-factor authentication (TOTP-based). SSO integration supports enterprise identity providers including Okta, Azure AD, Google Workspace, and OneLogin through standard OIDC Authorization Code Flow with PKCE. The system supports Just-In-Time (JIT) user provisioning where new users are automatically created on first SSO login, with group memberships mapped from IdP assertions to SynthForge AI roles. Session management uses JWT access tokens (15-minute expiry) with refresh token rotation, and all sessions are tracked in the database for audit purposes with configurable maximum concurrent sessions.

7.2 Role-Based Access Control

The RBAC system defines four built-in roles with configurable permissions at the workspace level. The Organization Owner has full control over the organization including billing, user management, and workspace creation. The Workspace Admin manages workspace settings, invites users, configures connections, and sets agent instructions. The Editor can create and modify datasets, run generations, and use the chat interface. The Viewer has read-only access to datasets, schemas, and generation results. Custom roles can be defined at the organization level with granular permissions for each resource type (workspaces, projects, databases, generations, exports). Permissions are evaluated through a policy engine that supports both positive (allow) and negative (deny) rules, with deny rules taking precedence to enable exclusion patterns within broader permissions.

Role	Workspaces	Projects	Generations	Billing	Users

Org Owner	Create/Delete/ Configure	Full Access	Full Access	Full Control	Manage All
Workspace Admin	Configure Assigned	Full Access	Full Access	View Only	Manage Workspace
Editor	View Assigned	Create/Modify	Create/Run	No Access	No Access
Viewer	View Assigned	View Only	View Results	No Access	No Access

Table 4: Role-Based Access Control Matrix

7.3 Compliance and Audit Logging

The audit logging system records every significant action in the platform, including user authentication events, data access events, generation operations, configuration changes, and administrative actions. Each audit log entry includes a timestamp, actor ID, action type, resource type and ID, before and after state (for mutations), IP address, user agent, and correlation ID linking related actions. Audit logs are stored in an append-only table with tamper-evident hashing (each entry includes a hash of the previous entry), making it possible to detect log modification or deletion. Logs are retained for a configurable period (default 7 years for enterprise) and can be exported to SIEM systems via webhook or API for integration with existing security monitoring infrastructure.

SOC 2 Type II compliance is achieved through a combination of technical controls (encryption at rest and in transit, access logging, vulnerability scanning, incident response procedures) and organizational controls (background checks, security training, policy documentation, annual penetration testing). HIPAA compliance includes Business Associate Agreement (BAA) support, PHI detection and handling in seed data, and audit controls that meet 45 CFR 164.312 requirements. GDPR compliance includes data processing agreements, right to erasure implementation, data portability export, and cross-border data transfer mechanisms.

7.4 Billing and Subscription Management

The billing system uses Stripe as the payment processor with a usage-based pricing model. Three subscription tiers are offered: Free (limited AI credits, basic export formats, community support), Professional ($49/month with $30 AI credits, all export formats, email support, advanced features), and Enterprise (custom pricing with pooled AI credits, SSO, RBAC, self-hosting, dedicated support, SLA). AI usage is metered in real-time with per-token tracking across all LLM providers, converted to a standardized credit system (1 credit = 1000 tokens). Overage billing follows a threshold-based model where usage beyond the included credits is charged at per-credit rates with configurable overage limits to prevent unexpected charges.

The subscription management system handles plan upgrades and downgrades with prorated billing, annual billing with discounts, enterprise license agreements with custom terms, and marketplace procurement (AWS Marketplace, Azure Marketplace). Usage analytics provide detailed breakdowns of AI credit consumption by workspace, project, agent, and generation type, with daily/weekly/monthly

trend charts and budget alerts. The system generates invoices with line-item detail for each billing period, including base subscription fees, AI usage overage charges, and any add-on service fees.

8. Database Schema and Infrastructure Design

The database schema is designed for multi-tenant SaaS operation with row-level security, comprehensive audit trails, and efficient query patterns for the most common access scenarios. PostgreSQL is the primary database, chosen for its strong ACID compliance, JSON support, full-text search capabilities, and mature ecosystem. The schema is organized into logical domains: authentication, workspaces, projects, datasets, generation, AI orchestration, and billing.

8.1 Core Entity Tables

Table	Purpose	Key Columns
organizations	Top-level billing entity	id, name, plan_type, stripe_customer_id, settings(jsonb)
users	Platform user accounts	id, email, password_hash, mfa_secret, last_login, created_at
workspaces	Tenant isolation boundary	id, org_id, name, slug, settings(jsonb), agent_instructions
workspace_members	User-workspace membership	user_id, workspace_id, role, joined_at
projects	Logical grouping of databases	id, workspace_id, name, description, created_at
databases	Synthetic database instances	id, project_id, name, schema_json(jsonb), db_type, created_at
db_connections	External database connections	id, workspace_id, name, connection_url(enc), db_type, config(jsonb)

Table 5: Core Entity Tables

8.2 Generation and AI Tables

Table	Purpose	Key Columns
conversations	Chat sessions with Data Agent	id, database_id, user_id, title, created_at, updated_at
messages	Individual chat messages	id, conversation_id, role, content, tokens_in, tokens_out, cost, created_at
generation_jobs	Async generation operations	id, database_id, conversation_id, status, plan_json(jsonb), progress, created_at

generation_steps	Individual steps in a job	id, job_id, agent_type, tool_name, input_json(jsonb), output_json(jsonb), status
datasets	Generated dataset versions	id, database_id, version, row_counts(jsonb), storage_path, seed, config(jsonb), created_at
exports	Export operation records	id, dataset_id, format, status, storage_path, file_size, created_at
agent_instructions	Custom agent behavior rules	id, workspace_id, database_id, instructions, priority, created_at
custom_skills	User-defined reusable patterns	id, workspace_id, name, description, skill_def(jsonb), created_at

Table 6: Generation and AI Tables

8.3 Billing and Audit Tables

Table	Purpose	Key Columns
subscriptions	Stripe subscription tracking	id, org_id, plan_type, stripe_sub_id, status, current_period_end
usage_records	Per-token usage metering	id, org_id, workspace_id, provider, model, tokens_in, tokens_out, cost, timestamp
invoices	Billing invoice records	id, org_id, stripe_invoice_id, amount, status, period_start, period_end
audit_logs	Tamper-evident audit trail	id, actor_id, action, resource_type, resource_id, before_state(jsonb), after_state(jsonb), prev_hash, timestamp
api_keys	API key management	id, user_id, name, key_hash, key_prefix, permissions(jsonb), last_used, expires_at
webhooks	Webhook endpoint config	id, org_id, url, events(jsonb), secret, active, created_at

Table 7: Billing and Audit Tables

8.4 Infrastructure Architecture

The infrastructure is designed for Kubernetes deployment with Helm charts for configuration management. The production environment runs on AWS (primary) with GCP and Azure as alternatives for enterprise customers requiring specific cloud providers. The core infrastructure includes an Application tier (FastAPI pods with Horizontal Pod Autoscaler, 3-50 replicas based on load), a Worker tier (Celery workers with priority-based queue segregation, 2-20 replicas), a Database tier (Amazon RDS PostgreSQL with Multi-AZ deployment, read replicas for analytics), a Cache tier (Amazon ElastiCache Redis with cluster mode for high availability), and a Storage tier (Amazon S3 with lifecycle policies for generated datasets and exports).

The Kubernetes cluster uses namespace-based isolation for multi-environment deployment (production, staging, development) with resource quotas and network policies enforcing traffic rules. Ingress is managed by Traefik with automatic TLS certificate provisioning via cert-manager and Let us Encrypt. The observability stack includes Prometheus for metrics collection, Grafana for dashboards, Loki for log aggregation, Jaeger for distributed tracing, and PagerDuty for alerting. CI/CD pipelines use GitHub Actions with automated testing, security scanning, and progressive deployment (canary releases with automatic rollback on error rate spikes).

Component	Technology	Configuration
API Servers	FastAPI + Gunicorn	3-50 pods, 1 CPU / 2GB RAM each, HPA on CPU/request-rate
Workers	Celery + Redis	2-20 pods, 2 CPU / 4GB RAM each, priority queues
Database	RDS PostgreSQL 16	db.r6g.xlarge, Multi-AZ, 2 read replicas, 100GB storage
Cache	ElastiCache Redis 7	cache.r6g.large, cluster mode, 3 shards, 1 replica each
Vector DB	Qdrant	2 pods, 4 CPU / 8GB RAM, persistent volume, 50GB storage
Object Storage	S3 + CloudFront	Versioned buckets, 30-day lifecycle to Glacier
Ingress	Traefik + cert-manager	TLS termination, rate limiting, WAF integration
Monitoring	Prometheus + Grafana	15s scrape interval, 30-day retention, custom dashboards

Table 8: Infrastructure Component Configuration

9. UI/UX Clone Specification

The UI/UX specification closely replicates the Tonic Fabricate user experience while introducing targeted improvements in discoverability, collaboration, and data visualization. The design system uses a consistent visual language built on TailwindCSS and shadcn/ui, with a neutral color palette that prioritizes data readability and reduces visual fatigue during extended data generation sessions. The interface follows a chat-first paradigm where all primary interactions occur through the conversational AI agent, supplemented by traditional UI controls for navigation, settings, and administration.

9.1 Landing Page

The landing page communicates the core value proposition: AI-powered synthetic data generation without requiring production data. The hero section features an animated demonstration of the chat interface generating a realistic dataset from a natural language prompt, with real-time data appearing in a simulated table preview. Key messaging emphasizes privacy-safety (no real data needed), speed (generate complete databases in minutes), and quality (relationally intact, statistically accurate). Below

the hero, feature cards highlight the Data Agent, Schema Upload, Mock APIs, and Workflow Automation. A pricing comparison table shows the three tiers with clear feature differentiation. Social proof includes customer logos, testimonial quotes, and compliance badges (SOC 2, HIPAA, GDPR).

9.2 Dashboard

The dashboard serves as the central hub after login, providing an overview of recent activity, usage metrics, and quick access to projects. The layout uses a sidebar navigation with collapsible sections for Workspaces, Projects, and Settings. The main content area displays: a welcome card with AI credit usage meter and recent conversations, a project grid showing each project with thumbnail previews of its databases, a usage chart showing daily AI credit consumption over the past 30 days, and a quick-action bar for creating new projects, uploading schemas, or starting a new conversation. The sidebar includes workspace switcher, user avatar menu, and notification bell with unread count.

9.3 Dataset Creation Wizard

The dataset creation wizard guides users through a four-step flow: Method Selection (Choose how to define your data: chat with AI, upload schema, upload sample data, or start from template), Schema Definition (Upload or define schema depending on method, with AI-assisted schema completion for partial inputs), Generation Configuration (Set row counts, specify data patterns or constraints, choose quality level, and set generation mode), and Preview and Generate (Preview a sample of generated data, adjust settings, and initiate full generation). Each step provides clear progress indication, back navigation, and contextual help. The wizard supports saving progress as a draft and resuming later, with automatic draft recovery on return visits.

9.4 AI Generation Studio (Chat Interface)

The AI Generation Studio is the primary interface for interacting with the Data Agent, implementing a chat-first UX pattern. The layout is split into three panels: a left sidebar showing conversation history and database navigation, a center panel for the chat conversation, and a right panel for data preview and schema visualization. The chat input supports text entry with Markdown formatting, file attachments (drag-and-drop or click), and slash commands for quick actions (/generate, /schema, /export, /undo, /help). Agent responses stream token-by-token with a typing indicator, and each response includes action buttons for copy, revert, and view SQL/JavaScript. When the agent generates data, the right panel updates in real-time with a table preview showing the new records highlighted. The cost indicator in the chat footer shows running token usage and estimated cost for the current conversation.

9.5 Schema Editor

The schema editor provides a visual interface for viewing and modifying database schemas. The main view is a table listing showing all tables with their columns, data types, constraints, and relationships. Clicking a table opens a detail panel with column definitions, foreign key relationships (visualized as a

mini entity-relationship diagram), and sample data. The editor supports inline editing of column names, types, and constraints with real-time validation. A visual ER diagram view shows the full database schema with interactive nodes (click to navigate to table detail) and edges representing foreign key relationships. Natural language schema modifications are supported through the chat interface, with changes reflected in the editor in real-time.

9.6 Prompt Playground

The prompt playground allows advanced users to test and refine generation prompts before using them in production workflows. The interface provides a split-pane editor with prompt template on the left and live preview on the right. Template variables are highlighted with syntax coloring and can be populated with test values through a form interface. The playground supports A/B testing by running the same prompt with different model configurations and displaying results side by side. Performance metrics (token usage, latency, quality score) are displayed below each result, and users can save tested prompts as custom skills for reuse in the Data Agent.

9.7 Team Management and Admin Dashboard

The team management interface provides organization and workspace administration capabilities. The organization settings page includes user management (invite, remove, role assignment), SSO configuration (connect identity providers, map groups to roles), billing management (subscription plan, payment methods, usage analytics), and compliance settings (audit log access, data retention policies, DSAR handling). The workspace settings page includes member management (invite to workspace, set permissions), connection management (add/edit/test database connections), agent instructions (workspace-level and database-level instructions), and workspace usage analytics. The admin dashboard provides a system-wide view for platform administrators with health monitoring, resource usage, and configuration management.

10. Production Code Examples

This section provides production-quality code examples for the core systems of SynthForge AI. Each example follows best practices for scalability, error handling, observability, and security. The code is structured for direct integration into the platform codebase with minimal modification.

10.1 AI Orchestration Layer

The AI orchestration layer provides the provider-agnostic abstraction that powers all LLM interactions in the platform. The implementation uses the Strategy pattern for provider selection, the Circuit Breaker pattern for fault tolerance, and an async-first design for high-throughput concurrent request handling.

```
# ai_orchestrator.py - Core AI Orchestration Layer from abc import ABC,
abstractmethod from dataclasses import dataclass, field from enum import Enum
from typing import AsyncIterator, Optional import asyncio, logging, time, hashlib
class ProviderType(Enum): OPENAI = "openai" CLAUDE = "claude" DEEPSEEK =
"deepseek" @dataclass class LLMRequest: messages: list[dict] model: Optional[str]
= None temperature: float = 0.7 max_tokens: int = 4096 response_format:
Optional[dict] = None tools: Optional[list[dict]] = None stream: bool = False
metadata: dict = field(default_factory=dict) @dataclass class LLMResponse:
content: str model: str provider: ProviderType tokens_in: int tokens_out: int
cost: float latency_ms: float tool_calls: Optional[list] = None class
LLMProvider(ABC): @abstractmethod async def complete(self, request: LLMRequest)
-> LLMResponse: ... @abstractmethod async def stream(self, request: LLMRequest)
-> AsyncIterator[str]: ... @abstractmethod def count_tokens(self, text: str) ->
int: ... @abstractmethod def estimate_cost(self, tokens_in: int, tokens_out: int)
-> float: ... class CircuitBreaker: def __init__(self, max_failures=5,
reset_timeout=60): self.max_failures = max_failures self.reset_timeout =
reset_timeout self.failure_count = 0 self.last_failure_time = 0 self.state =
"closed" # closed, open, half_open def record_success(self): self.failure_count =
0 self.state = "closed" def record_failure(self): self.failure_count += 1
self.last_failure_time = time.time() if self.failure_count >= self.max_failures:
self.state = "open" def can_execute(self) -> bool: if self.state == "closed":
return True if self.state == "open": if time.time() - self.last_failure_time >
self.reset_timeout: self.state = "half_open" return True return False return True
# half_open allows limited requests class AIOrchestrator: def __init__(self):
self.providers: dict[ProviderType, LLMProvider] = {} self.circuit_breakers:
dict[ProviderType, CircuitBreaker] = {} self.routing_config = {} # per-tenant
routing preferences def register_provider(self, provider_type: ProviderType,
provider: LLMProvider): self.providers[provider_type] = provider
self.circuit_breakers[provider_type] = CircuitBreaker() async def complete(self,
request: LLMRequest, tenant_id: str = None) -> LLMResponse: providers =
self._get_routing_order(tenant_id) last_error = None for provider_type in
providers: cb = self.circuit_breakers[provider_type] if not cb.can_execute():
continue try: response = await self.providers[provider_type].complete(request)
cb.record_success() return response except Exception as e: cb.record_failure()
last_error = e logging.warning(f"Provider {provider_type} failed: {e}") raise
RuntimeError(f"All providers failed. Last error: {last_error}") def
_get_routing_order(self, tenant_id: str = None) -> list[ProviderType]: #
Priority: tenant config > default (cost-optimized) if tenant_id and tenant_id in
self.routing_config: return self.routing_config[tenant_id].get("priority_order",
[ProviderType.DEEPSEEK, ProviderType.CLAUDE, ProviderType.OPENAI]) return
[ProviderType.DEEPSEEK, ProviderType.CLAUDE, ProviderType.OPENAI]
```

10.2 Synthetic Data Pipeline

The synthetic data pipeline orchestrates the end-to-end generation process from schema analysis through export. The implementation uses a pipeline pattern where each stage transforms the data and passes results to the next stage, with comprehensive error handling and checkpoint-based recovery for long-running batch operations.

```
# synthetic_pipeline.py - Synthetic Data Generation Pipeline from dataclasses
import dataclass, field from enum import Enum from typing import Optional import
asyncio, json, logging class PipelineStage(Enum): SCHEMA_ANALYSIS =
```

```
"schema_analysis" DATA_PLANNING = "data_planning" GENERATION = "generation"
VALIDATION = "validation" EXPORT = "export" @dataclass class SchemaInfo: tables:
list[dict] relationships: list[dict] constraints: list[dict] generation_order:
list[str] = field(default_factory=list) @dataclass class GenerationPlan:
table_plans: dict[str, dict] # table_name -> generation config total_rows: int
estimated_cost: float estimated_time_seconds: float @dataclass class
GenerationResult: table_data: dict[str, list[dict]] quality_score: float
validation_report: dict tokens_used: int cost: float class SyntheticDataPipeline:
def __init__(self, orchestrator, schema_agent, quality_agent, generation_agent,
validation_agent): self.orchestrator = orchestrator self.agents = { "schema":
schema_agent, "quality": quality_agent, "generation": generation_agent,
"validation": validation_agent, } self.checkpoints = {} async def execute(self,
schema_input: str, format: str, row_count: int, seed: Optional[int] = None,
callback=None) -> GenerationResult: # Stage 1: Schema Analysis schema = await
self._run_stage( PipelineStage.SCHEMA_ANALYSIS, self.agents["schema"].analyze,
schema_input, format) # Stage 2: Data Planning plan = await self._run_stage(
PipelineStage.DATA_PLANNING, self.agents["schema"].plan_generation, schema,
row_count) # Stage 3: Generation result = await self._run_stage(
PipelineStage.GENERATION, self.agents["generation"].generate, schema, plan, seed)
# Stage 4: Validation validation = await self._run_stage(
PipelineStage.VALIDATION, self.agents["validation"].validate, result, schema) if
not validation["passed"]: result = await self._remediate(result, validation,
schema) # Stage 5: Export export_path = await self._run_stage(
PipelineStage.EXPORT, self._export_data, result, format) if callback: await
callback("complete", {"path": export_path}) return result async def
_run_stage(self, stage, func, *args): try: result = await func(*args)
self.checkpoints[stage.value] = result return result except Exception as e:
logging.error(f"Pipeline stage {stage.value} failed: {e}") if stage.value in
self.checkpoints: logging.info(f"Resuming from checkpoint for {stage.value}")
raise async def _remediate(self, result, validation, schema): issues =
validation.get("issues", []) for issue in issues: table = issue["table"] rows =
issue["failing_rows"] fixed = await self.agents["generation"].regenerate_rows(
schema, table, rows) result.table_data[table] = self._merge_fixed_rows(
result.table_data[table], fixed, rows) return result
```

10.3 Multi-Agent Coordination

The multi-agent coordination system manages communication between specialized agents through a structured message bus. The Orchestrator Agent dispatches tasks to specialized agents, collects results, and makes routing decisions based on the current generation context and agent capabilities.

```
# agent_coordinator.py - Multi-Agent Coordination from dataclasses import
dataclass, field from enum import Enum from typing import Optional, Callable,
Awaitable import asyncio, json, uuid from datetime import datetime class
MessageType(Enum): REQUEST = "request" RESPONSE = "response" NOTIFICATION =
"notification" ERROR = "error" @dataclass class AgentMessage: id: str =
field(default_factory=lambda: str(uuid.uuid4())) sender: str = "" recipient: str
= "" # "" for broadcast msg_type: MessageType = MessageType.REQUEST payload: dict
= field(default_factory=dict) correlation_id: Optional[str] = None timestamp: str
= field(default_factory=lambda: datetime.utcnow().isoformat()) class
AgentCoordinator: def __init__(self, orchestrator): self.orchestrator =
orchestrator self.agents: dict[str, object] = {} self.handlers: dict[str,
```

```
list[Callable]] = {} self.pending: dict[str, asyncio.Future] = {} def
register_agent(self, name: str, agent): self.agents[name] = agent
agent.set_coordinator(self) async def send(self, message: AgentMessage) ->
Optional[dict]: if message.msg_type == MessageType.REQUEST: future =
asyncio.get_event_loop().create_future() self.pending[message.id] = future await
self._dispatch(message) return await asyncio.wait_for(future, timeout=300) await
self._dispatch(message) return None async def _dispatch(self, message:
AgentMessage): if message.recipient and message.recipient in self.agents: agent =
self.agents[message.recipient] response = await agent.handle_message(message) if
response and message.id in self.pending:
self.pending[message.id].set_result(response) elif not message.recipient: #
broadcast for agent in self.agents.values(): await agent.handle_message(message)
async def coordinate_generation(self, schema_input, config): # Step 1: Analyze
schema schema_result = await self.send(AgentMessage( sender="orchestrator",
recipient="schema_agent", msg_type=MessageType.REQUEST, payload={"action":
"analyze", "input": schema_input})) # Step 2: Plan generation plan_result = await
self.send(AgentMessage( sender="orchestrator", recipient="schema_agent",
msg_type=MessageType.REQUEST, payload={"action": "plan", "schema":
schema_result})) # Step 3: Generate data (parallel for independent tables)
gen_tasks = [] for table_plan in plan_result["table_plans"]:
gen_tasks.append(self.send(AgentMessage( sender="orchestrator",
recipient="generation_agent", msg_type=MessageType.REQUEST, payload={"action":
"generate", "plan": table_plan}))) gen_results = await asyncio.gather(*gen_tasks)
# Step 4: Validate (parallel quality + compliance) val_tasks = [
self.send(AgentMessage( sender="orchestrator", recipient="quality_agent",
payload={"action": "validate", "data": gen_results})), self.send(AgentMessage(
sender="orchestrator", recipient="security_agent", payload={"action":
"compliance_check", "data": gen_results})) ] quality_result, compliance_result =
await asyncio.gather(*val_tasks) return { "data": gen_results, "quality":
quality_result, "compliance": compliance_result }
```

10.4 Authentication and RBAC

The authentication system implements JWT-based stateless authentication with SSO support through OIDC. The RBAC engine evaluates permissions through a policy system that supports both role-based and attribute-based access control.

```
# auth_rbac.py - Authentication and RBAC System from dataclasses import
dataclass, field from enum import Enum from typing import Optional import
hashlib, secrets, time, jwt class Permission(Enum): WORKSPACE_CREATE =
"workspace:create" WORKSPACE_DELETE = "workspace:delete" PROJECT_CREATE =
"project:create" PROJECT_READ = "project:read" PROJECT_WRITE = "project:write"
GENERATION_CREATE = "generation:create" GENERATION_READ = "generation:read"
EXPORT_CREATE = "export:create" BILLING_READ = "billing:read" BILLING_MANAGE =
"billing:manage" USER_MANAGE = "user:manage" ADMIN_ACCESS = "admin:access"
ROLE_PERMISSIONS = { "org_owner": set(Permission), # all permissions
"workspace_admin": { Permission.PROJECT_CREATE, Permission.PROJECT_READ,
Permission.PROJECT_WRITE, Permission.GENERATION_CREATE,
Permission.GENERATION_READ, Permission.EXPORT_CREATE, Permission.BILLING_READ,
Permission.USER_MANAGE, }, "editor": { Permission.PROJECT_READ,
Permission.PROJECT_WRITE, Permission.GENERATION_CREATE,
Permission.GENERATION_READ, Permission.EXPORT_CREATE, }, "viewer": {
```

```
Permission.PROJECT_READ, Permission.GENERATION_READ, }, } @dataclass class
UserContext: user_id: str org_id: str workspace_id: Optional[str] = None roles:
list[str] = field(default_factory=list) api_key_id: Optional[str] = None class
AuthService: def __init__(self, jwt_secret: str, jwt_expiry: int = 900):
self.jwt_secret = jwt_secret self.jwt_expiry = jwt_expiry # 15 minutes def
create_token(self, user_ctx: UserContext) -> str: payload = { "sub":
user_ctx.user_id, "org": user_ctx.org_id, "ws": user_ctx.workspace_id, "roles":
user_ctx.roles, "type": "api_key" if user_ctx.api_key_id else "user", "iat":
int(time.time()), "exp": int(time.time()) + self.jwt_expiry, } return
jwt.encode(payload, self.jwt_secret, algorithm="HS256") def verify_token(self,
token: str) -> Optional[UserContext]: try: payload = jwt.decode(token,
self.jwt_secret, algorithms=["HS256"]) return UserContext(
user_id=payload["sub"], org_id=payload["org"], workspace_id=payload.get("ws"),
roles=payload.get("roles", []), api_key_id=payload.get("type") == "api_key", )
except jwt.ExpiredSignatureError: return None class RBACEngine: def
has_permission(self, user_ctx: UserContext, permission: Permission,
resource_workspace_id: str = None) -> bool: for role in user_ctx.roles: if
permission in ROLE_PERMISSIONS.get(role, set()): if permission in
(Permission.WORKSPACE_CREATE, Permission.BILLING_MANAGE,
Permission.ADMIN_ACCESS): return True # org-level permissions if
resource_workspace_id == user_ctx.workspace_id: return True return False
```

10.5 Streaming Architecture and WebSocket Updates

The streaming architecture supports real-time token delivery for LLM responses and WebSocket-based status updates for generation jobs. The implementation uses Server-Sent Events for token streaming (simpler than WebSocket for unidirectional data flow) and WebSocket for bidirectional communication (job status, notifications, collaborative editing).

```
# streaming.py - SSE and WebSocket Streaming Architecture from fastapi import
APIRouter, WebSocket, Depends, Query from fastapi.responses import
StreamingResponse from typing import AsyncIterator import asyncio, json router =
APIRouter() @router.get("/api/v1/chat/{conversation_id}/stream") async def
stream_chat(conversation_id: str, message: str, user=Depends(get_current_user)):
"""Stream LLM token responses via Server-Sent Events.""" async def
event_generator() -> AsyncIterator[str]: orchestrator = get_orchestrator()
request = LLMRequest( messages=await build_messages(conversation_id, message),
stream=True, metadata={"user_id": user.user_id}) tokens_used = 0 async for token
in orchestrator.stream(request, tenant_id=user.org_id): tokens_used += 1
event_data = json.dumps({ "token": token, "conversation_id": conversation_id })
yield f"data: {event_data}\n\n" # Send completion event with usage stats
completion_data = json.dumps({ "type": "complete", "tokens_used": tokens_used,
"cost": calculate_cost(tokens_used) }) yield f"data: {completion_data}\n\n"
return StreamingResponse( event_generator(), media_type="text/event-stream",
headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"}) class
ConnectionManager: def __init__(self): self.active: dict[str, WebSocket] = {} #
user_id -> WebSocket async def connect(self, websocket: WebSocket, user_id: str):
await websocket.accept() self.active[user_id] = websocket def disconnect(self,
user_id: str): self.active.pop(user_id, None) async def send_update(self,
user_id: str, event: dict): ws = self.active.get(user_id) if ws: await
ws.send_json(event) manager = ConnectionManager()
@router.websocket("/ws/{user_id}") async def websocket_endpoint(websocket:
```

```
WebSocket, user_id: str, token: str = Query(...)): user =
AuthService(JWT_SECRET).verify_token(token) if not user or user.user_id !=
user_id: await websocket.close(code=4001) return await manager.connect(websocket,
user_id) try: while True: data = await websocket.receive_json() # Handle incoming
WebSocket messages (e.g., subscribe to job) if data.get("action") ==
"subscribe_job": # Job progress updates will be pushed via send_update pass
except Exception: manager.disconnect(user_id) # In worker: publish job updates
async def publish_job_update(user_id: str, job_id: str, progress: float, status:
str): await manager.send_update(user_id, { "type": "job_update", "job_id":
job_id, "progress": progress, "status": status, })
```

10.6 Schema Inference Engine

The schema inference engine parses multiple input formats and produces a normalized internal schema representation. The implementation supports SQL DDL, Prisma schemas, Swagger/OpenAPI, CSV headers, and JSON Schema, with AI-powered fallback for incomplete or ambiguous schemas.

```
# schema_inference.py - Multi-Format Schema Inference from dataclasses import
dataclass, field from typing import Optional import json, re, logging @dataclass
class ColumnDef: name: str data_type: str nullable: bool = True unique: bool =
False primary_key: bool = False foreign_key: Optional[dict] = None # {table,
column} default: Optional[str] = None check_constraint: Optional[str] = None
semantic_type: Optional[str] = None # email, phone, name, etc. description:
Optional[str] = None @dataclass class TableDef: name: str columns:
list[ColumnDef] = field(default_factory=list) primary_key: list[str] =
field(default_factory=list) foreign_keys: list[dict] =
field(default_factory=list) unique_constraints: list[list[str]] =
field(default_factory=list) row_count_hint: Optional[int] = None @dataclass class
InferredSchema: tables: list[TableDef] = field(default_factory=list) format: str
= "" raw_input: str = "" class SchemaInferrer: def infer(self, input_text: str,
format_hint: str = None) -> InferredSchema: detected = format_hint or
self._detect_format(input_text) if detected == "sql_ddl": return
self._parse_sql_ddl(input_text) elif detected == "prisma": return
self._parse_prisma(input_text) elif detected == "swagger": return
self._parse_swagger(input_text) elif detected == "csv": return
self._parse_csv(input_text) elif detected == "json_schema": return
self._parse_json_schema(input_text) else: raise ValueError(f"Unsupported format:
{detected}") def _detect_format(self, text: str) -> str: text_lower =
text.strip().lower() if any(kw in text_lower for kw in ["create table", "create
index"]): return "sql_ddl" if "model " in text_lower and "datasource" in
text_lower: return "prisma" if "openapi" in text_lower or "swagger" in
text_lower: return "swagger" if text.strip().startswith("{") and "type" in
text_lower: return "json_schema" if "," in text and "\n" in text: return "csv"
return "unknown" def _parse_sql_ddl(self, ddl: str) -> InferredSchema: schema =
InferredSchema(format="sql_ddl", raw_input=ddl) table_pattern = re.compile(
r"CREATE\s+TABLE\s+(?:IF\s+NOT\s+EXISTS\s+)?" r"[\'"]?(\w+)[\'"]?\s*\((.*?)\);",
re.IGNORECASE | re.DOTALL) for match in table_pattern.finditer(ddl): table_name =
match.group(1) body = match.group(2) table = TableDef(name=table_name) for line
in self._split_columns(body): col = self._parse_column_def(line) if col:
table.columns.append(col) schema.tables.append(table)
self._resolve_foreign_keys(schema) return schema
```

11. Deployment and Monetization Strategy

The deployment and monetization strategy is designed for a venture-scale SaaS business with clear paths to enterprise revenue. The platform supports both managed SaaS deployment and enterprise self-hosting, with a usage-based pricing model that aligns revenue with customer value. The strategy addresses cloud architecture, cost optimization, scaling roadmap, and enterprise sales motion.

11.1 SaaS Deployment Architecture

The SaaS deployment runs on AWS with a multi-region architecture for low-latency global access. The primary region (us-east-1) hosts the application tier, database, and cache, while a secondary region (eu-west-1) provides disaster recovery and serves European customers with data residency compliance. The application tier runs on Elastic Kubernetes Service (EKS) with cluster autoscaler for dynamic capacity management. Amazon RDS PostgreSQL provides the primary database with Multi-AZ deployment, automated backups (35-day retention), and cross-region read replicas for analytics. Amazon ElastiCache Redis provides the caching and queue layer with cluster mode for high availability. Amazon S3 stores generated datasets and exports with lifecycle policies that transition older datasets to Glacier after 90 days. Amazon CloudFront serves as the CDN for static assets and dataset downloads, with edge caching for frequently accessed schemas and templates.

11.2 Pricing Model

Feature	Free	Professional ($49/mo)	Enterprise (Custom)
AI Credits/Month	50 credits	300 credits	Custom (pooled)
Overage Handling	Paused until reset	Pay-as-you-go ($0.02/credit)	Quarterly billing
Workspaces	1	5	Unlimited
Export Formats	CSV, JSON, SQLite	All structured formats	All formats including PDF/DOCX
Database Connections	1	10	Unlimited
Mock APIs	2 endpoints	50 endpoints	Unlimited
SSO/SAML	No	No	Yes (OIDC, SAML 2.0)
RBAC	No	Basic (2 roles)	Custom roles and permissions
Self-Hosting	No	No	Yes (Docker/K8s)

BYO LLM Key	No	No	Yes
Support	Community	Email (24h SLA)	Dedicated + Slack (4h SLA)
Audit Logs	7 days	90 days	7 years + SIEM export

Table 9: Pricing Tier Comparison

11.3 Usage Metering and Cost Optimization

Usage metering tracks AI credit consumption in real-time across all LLM providers, converting per-token costs to a standardized credit system where 1 credit equals approximately 1000 tokens (adjusted for model tier). The metering system uses a write-ahead log pattern where token usage is recorded immediately upon LLM response receipt, with periodic aggregation for billing purposes. Cost optimization strategies include: prompt caching (avoid regenerating identical prompts within a session), deterministic generation for simple data types (names, addresses, dates), batch processing for large datasets (reducing per-record LLM overhead), and intelligent model routing (using cost-effective models for simple tasks). GPU optimization is achieved through serverless GPU inference endpoints (AWS Inferentia2) for embedding generation and NER model inference, with automatic scaling to zero during low-usage periods.

11.4 Scaling Roadmap

The scaling roadmap defines three phases aligned with business growth. Phase 1 (MVP, months 1-6) focuses on core data generation with the chat interface, supporting SQL DDL and CSV schemas, basic export formats, and the Free and Professional tiers. The infrastructure targets 100 concurrent users with 3 application pods, 2 worker pods, and a single-region deployment. Phase 2 (Growth, months 7-12) expands to full schema support (Prisma, Swagger, JSON Schema), Mock APIs, Workflow Automation, Custom Skills, and the Enterprise tier with SSO and self-hosting. Infrastructure scales to 1000 concurrent users with multi-region deployment, read replicas, and auto-scaling. Phase 3 (Scale, months 13-24) adds advanced features including Live Connect, database profiling, unstructured document generation, prompt playground A/B testing, and marketplace integrations. Infrastructure scales to 10,000+ concurrent users with global CDN, multi-cloud deployment options, and dedicated enterprise infrastructure.

12. Final Deliverables and Project Plan

This section consolidates the complete set of deliverables for the SynthForge AI platform, including the file and folder structure, MVP plan, and production scaling timeline. These deliverables represent the full scope of work required to build a production-grade, venture-scale competitor to Tonic Fabricate.

12.1 File and Folder Structure

```
synthforge-ai/ apps/ web/ # Next.js 15+ frontend src/ app/ # App Router pages (auth)/ # Auth pages (login, register, sso) (dashboard)/ # Dashboard layout + pages (studio)/ # AI Generation Studio layout api/ # Next.js API routes components/ chat/ # Chat interface components schema/ # Schema editor components dataset/ # Dataset visualizer components workflow/ # Workflow builder components common/ # Shared UI components (shadcn/ui) lib/ ai/ # AI SDK client integration stores/ # Zustand state stores hooks/ # Custom React hooks styles/ # TailwindCSS configuration public/ # Static assets api/ # FastAPI backend app/ api/v1/ # API route modules auth/ # Authentication endpoints projects/ # Project CRUD databases/ # Database management generation/ # Generation orchestration agents/ # Agent communication export/ # Export pipeline billing/ # Usage and billing core/ # Shared utilities config.py # Configuration management security.py # Auth, RBAC, encryption database.py # SQLAlchemy session management models/ # SQLAlchemy ORM models services/ # Business logic layer schemas/ # Pydantic request/response schemas workers/ # Celery worker definitions alembic/ # Database migrations packages/ ai-orchestrator/ # LLM provider abstraction providers/ # OpenAI, Claude, DeepSeek routing/ # Model routing engine caching/ # Prompt caching layer synthetic-engine/ # Synthetic data generation generators/ # Deterministic + AI generators validators/ # Constraint and quality validation exporters/ # Format-specific exporters parsers/ # Schema parsers (SQL, Prisma, etc.) agent-system/ # Multi-agent coordination agents/ # Specialized agent implementations messaging/ # Agent communication bus memory/ # Agent memory management shared/ # Shared utilities types/ # TypeScript/Python type definitions utils/ # Common utility functions infra/ docker/ # Dockerfile + docker-compose k8s/ # Kubernetes manifests + Helm charts terraform/ # AWS infrastructure as code ci/ # GitHub Actions workflows docs/ api/ # OpenAPI specifications architecture/ # Architecture decision records
```

12.2 MVP Plan (Phase 1: Months 1-6)

Sprint	Duration	Deliverables
Sprint 1-2	Weeks 1-4	Project scaffolding, auth system, workspace CRUD, basic UI shell
Sprint 3-4	Weeks 5-8	AI orchestrator with OpenAI + Claude, chat interface MVP, streaming responses
Sprint 5-6	Weeks 9-12	SQL DDL parser, synthetic generation agent, basic data generation (single table)
Sprint 7-8	Weeks 13-16	Multi-table generation with FK support, dataset visualizer, CSV/JSON export
Sprint 9-10	Weeks 17-20	Schema editor UI, file upload (CSV, SQL), iterative refinement via chat
Sprint 11-12	Weeks 21-24	Billing integration, usage metering, Free/Pro tiers, landing page, beta launch

Table 10: MVP Sprint Plan

12.3 Production Scaling Plan

The production scaling plan follows a progressive approach aligned with the three-phase roadmap. In Phase 1 (MVP), the system handles up to 100 concurrent users with a single-region deployment on a single EKS cluster, RDS PostgreSQL with one read replica, and ElastiCache Redis in cluster mode with 3 shards. Monthly infrastructure cost is estimated at $2,500-3,500. In Phase 2 (Growth), the system scales to 1,000 concurrent users with multi-region deployment, additional read replicas for analytics workloads, and auto-scaling application and worker tiers. Monthly infrastructure cost is estimated at $8,000-15,000. In Phase 3 (Scale), the system handles 10,000+ concurrent users with global CDN, multi-cloud deployment options for enterprise customers, dedicated infrastructure for high-volume tenants, and advanced caching layers. Monthly infrastructure cost is estimated at $25,000-50,000, offset by revenue from enterprise contracts with annual values of $50,000-500,000.

Key scaling metrics include: API response time p99 less than 500ms for non-generation endpoints, generation throughput of 10,000 rows per minute for batch operations, chat response time-to-first-token less than 1 second for streaming, system availability of 99.9% (measured monthly), and database connection pool utilization below 70% under peak load. The monitoring stack (Prometheus + Grafana + Loki + Jaeger) provides end-to-end observability with custom dashboards for each service, automated alerting for SLO violations, and distributed tracing for debugging complex multi-agent workflows.